EXPERIMENT 4

Implement the Activity Selection Problem Using the Greedy Approach

Name	HIMANSHU RAJ
Batch	B1 CCVT
SAP ID	590026147
Submitted To	Arjun Abhishek

B.Tech CSE – Cloud Computing & Virtualization (CCVT)

1. Aim

To implement the Activity Selection Problem using the Greedy approach in C and select the maximum number of mutually compatible activities.

2. Objectives

• Understand the Greedy algorithmic approach.

• Sort activities in ascending order of finish time.

• Select the maximum number of non-overlapping activities.

• Analyze the time and space complexity of the algorithm.

3. Theory

The Activity Selection Problem involves selecting the maximum number of activities that can be performed by a single person or resource, when only one activity can be performed at a time. Each activity has a start time and a finish time.

The greedy strategy selects the compatible activity that finishes earliest. This leaves as much time as possible for the remaining activities. Two activities are compatible when the next activity starts at or after the finish time of the previously selected activity.

4. Algorithm

1. Read the start and finish times of all activities.

2. Sort the activities in ascending order of finish time.

3. Select the first activity.

4. For each remaining activity, check whether its start time is greater than or equal to the finish time of the last selected activity.

5. If compatible, select the activity and update the finish time of the last selected activity.

6. Display the selected activities and the total count.

5. Sample Input and Output

Number of activities: 6

Activity 1: Start = 1, Finish = 3

Activity 2: Start = 2, Finish = 5

Activity 3: Start = 4, Finish = 6

Activity 4: Start = 6, Finish = 8

Activity 5: Start = 5, Finish = 7

Activity 6: Start = 8, Finish = 9

The program selected Activities 1, 3, 4, and 6. The maximum number of compatible activities selected was 4.

6. Complexity Analysis

Operation	Time Complexity
Bubble sort by finish time	$O(n^2)$
Greedy selection pass	$O(n)$
Overall implementation	$O(n^2)$
Auxiliary space	O(1) apart from stored activities

7. Performance and Greedy Choice

The program sorts activities by increasing finish time and then scans the sorted list once. For the sample input, it selects Activity 1 (1–3), Activity 3 (4–6), Activity 4 (6–8), and Activity 6 (8–9). Each selected activity begins at or after the previous selected activity finishes. The greedy selection pass is $O(n)$, while bubble sorting makes the overall implementation $O(n^2)$.

8. Result

The Activity Selection Problem was implemented successfully in C using the Greedy approach. For the given six activities, the algorithm selected four mutually compatible activities: 1, 3, 4, and 6.

9. Screenshots / Evidence

The following screenshots show the implementation and successful execution in Visual Studio Code. Source code is omitted from the report, following the format of the previous experiment PDFs.

Screenshot 1

exp4.c > ...

```c

#include <stdio.h>

#define MAX 100

struct Activity {
    int start;
    int finish;
    int id;
};

// Sort activities by finish time
Windsurf: Refactor | Explain | ×
void sortActivities(struct Activity a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j].finish > a[j + 1].finish) {
                struct Activity temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

// Greedy activity selection
Windsurf: Refactor | Explain | ×
void activitySelection(struct Activity a[], int n) {
    int count = 0;
    int lastFinish = -1;

    printf("\nSelected Activities:\n");

    for (int i = 0; i < n; i++) {
        if (count == 0 || a[i].start >= lastFinish) {
            printf("Activity %d: Start = %d, Finish = %d\n",
                   a[i].id, a[i].start, a[i].finish);

            lastFinish = a[i].finish;
            count++;
        }
    }

    printf("\nMaximum number of activities selected: %d\n", count);
}

```

Screenshot 1: Activity structure, sorting activities by finish time, and greedy selection logic.

Screenshot 2

```c
int main(void) {
    struct Activity a[MAX];
    int n;

    printf("Enter number of activities (1-%d): ", MAX);

    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
        printf("Invalid number of activities!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        a[i].id = i + 1;

        printf("Enter start and finish time for Activity %d: ", i + 1);

        if (scanf("%d %d", &a[i].start, &a[i].finish) != 2 ||
            a[i].start < 0 || a[i].finish <= a[i].start) {
            printf("Invalid activity time!\n");
            return 1;
        }
    }

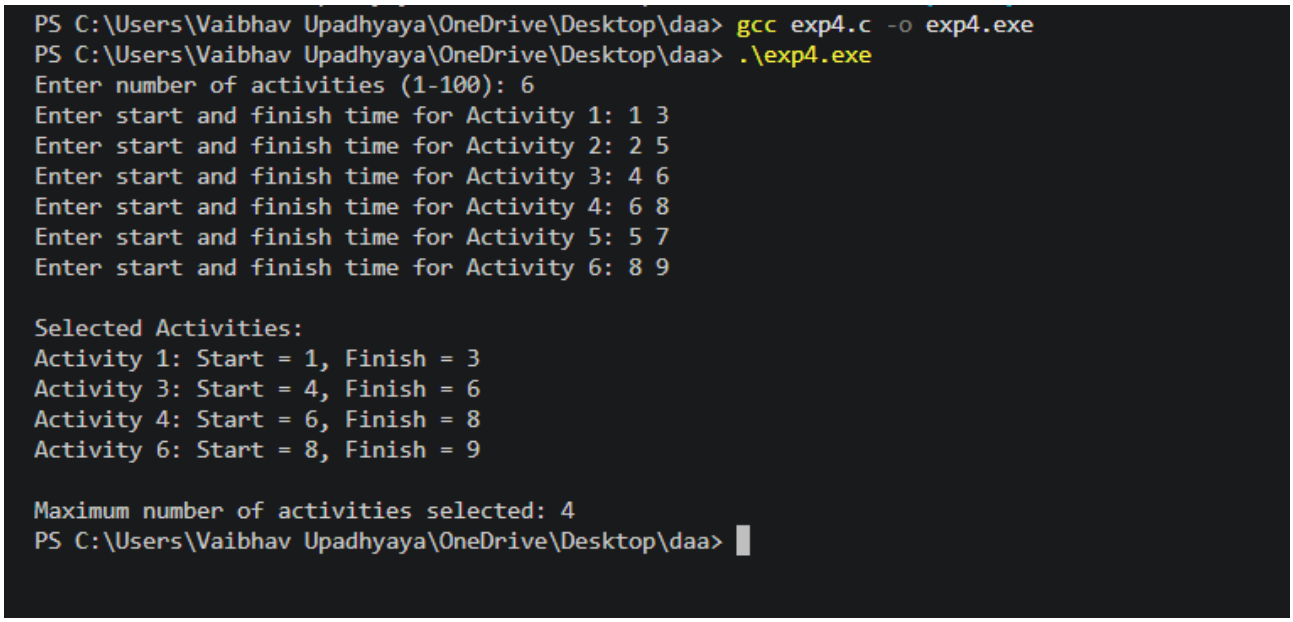
    sortActivities(a, n);

    activitySelection(a, n);

    return 0;
}
```

Screenshot 2: Main function, input validation, and calls to sorting and activity selection.

Screenshot 3



Screenshot 3: Successful program execution showing the four selected activities.